Chapter 15 — HashMap — Complete Deep Dive

> **Interview Importance: ⭐⭐⭐⭐⭐ — One of the most important Java interview topics**

`HashMap` is the implementation you should understand **internally**, not just at API level.

The central idea:

> **HashMap converts a key into a hash, uses that hash to locate a bucket, and then uses equality to identify the exact key.**

We'll build the internals without repeating the basic `Map` concepts from Chapter 14.

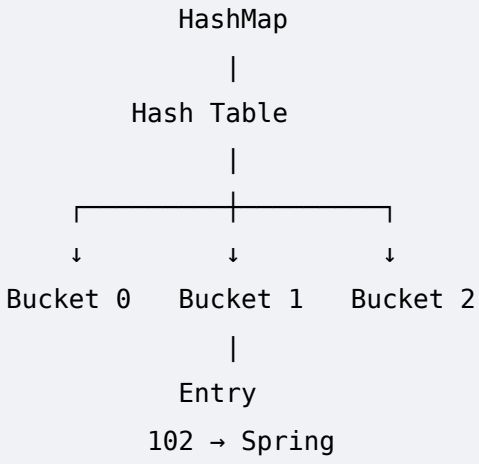
---

1. HashMap at a Glance

```
Map<Integer, String> map = new HashMap<>();

map.put(101, "Java");
map.put(102, "Spring");
```

Conceptually:



Typical complexity:

Operation	Average	Worst/Pathological
`put()`	O(1)	O(log n) with tree

		bins
`get()`	O(1)	O(log n) with tree bins
`remove()`	O(1)	O(log n) with tree bins
`containsKey()`	O(1)	O(log n) with tree bins

The O(1) figure assumes good hash distribution.

---

2. What Does HashMap Actually Store?

Internally, HashMap maintains an array called the table.

Conceptually:

```
Node<K,V>[] table;
```

Each occupied bucket contains a chain/tree of nodes.

A simplified Node looks like:

```
static class Node<K,V> {
    final int hash;
    final K key;
    V value;
    Node<K,V> next;
}
```

So each entry contains:

```
hash
key
value
next
```

The `next` reference allows multiple entries to exist in the same bucket.

---

3. HashMap Internal Structure

Suppose:

```
map.put("A", 100);
map.put("B", 200);
map.put("C", 300);
```

Conceptually:

```
table
  |
  +-- bucket 0 → null
  |
  +-- bucket 1 → [A,100]
  |
  +-- bucket 2 → [B,200]
  |
  +-- bucket 3 → [C,300]
```

If two keys land in the same bucket:

```
bucket 2
   ↓
[A,100] → [B,200]
```

That's a **hash collision**.

---

4. The HashMap `put()` Journey

When you execute:

```
map.put(key, value);
```

Think:

```
put()
 ↓
Calculate hash
 ↓
Calculate bucket index
```

```
 ↓
Check bucket
 ↓
Empty?
 ├── YES → insert
 └── NO
       ↓
   Compare existing keys
       ↓
   Same key?
   ├── YES → replace value
   └── NO  → collision handling
```

If the bucket becomes sufficiently crowded, it may be converted from a linked structure to a tree.

---

5. Step 1 — Calculate Hash

For a key:

```
String key = "Java";
```

HashMap obtains the key's hash code:

```
key.hashCode();
```

But modern HashMap doesn't simply use that value directly.

It performs additional bit mixing.

---

6. Hash Spreading

Conceptually, Java 8+ HashMap uses:

```
h ^ (h >>> 16)
```

The implementation's hash method is essentially:

```
static final int hash(Object key) {
    int h;
    return (key == null)
        ? 0
```

```
                : (h = key.hashCode()) ^ (h >>> 16);
    }
```

Why?

Because the bucket index uses only part of the hash.

Mixing high bits into low bits helps improve distribution.

---

7. Why Mix the Hash?

Suppose the bucket index depends heavily on low bits.

If a key's `hashCode()` has poor variation in those bits, many keys can land in the same bucket.

Hash spreading:

```
Original hash
      ↓
Mix high bits into low bits
      ↓
Better bucket distribution
```

This reduces unnecessary collisions.

---

8. Step 2 — Calculate Bucket Index

For a table of size `n`, HashMap conceptually calculates:

```
index = (n - 1) & hash;
```

This is one of the most frequently asked HashMap questions.

Why not:

```
hash % n
```

?

Because bitwise AND is efficient when `n` is a power of two.

---

9. Why HashMap Capacity Is a Power of 2

Typical capacities:

```
16
32
64
128
256
...
```

Suppose:

```
n = 16
```

Then:

```
n - 1 = 15
```

Binary:

```
15 = 1111
```

Therefore:

```
hash & 1111
```

efficiently extracts the relevant lower bits.

This is the basis for:

```
(hash & (n - 1))
```

---

10. Why Not Any Arbitrary Capacity?

If:

```
n = 16
```

then:

```
n - 1 = 15 = 1111
```

This gives a clean bit mask.

If:

```
n = 10
```

then:

```
n - 1 = 9 = 1001
```

The distribution properties are not equivalent.

HashMap therefore uses power-of-two table sizes.

---

11. Important Null-Key Special Case

HashMap permits one `null` key.

Conceptually:

```
map.put(null, "Hello");
```

HashMap assigns:

```
hash = 0
```

to a null key.

So null handling doesn't require calling:

```
null.hashCode()
```

which obviously isn't possible.

---

12. Step 3 — Find the Bucket

After calculating:

```
index = (n - 1) & hash
```

HashMap accesses:

```
table[index]
```

If:

```
table[index] == null
```

the entry can be inserted directly.

If not, collision handling begins.

---

13. Collision

A collision occurs when different keys map to the same bucket.

For example:

```
Key A → bucket 5
Key B → bucket 5
```

Even if:

```
A.equals(B) == false
```

they can still share a bucket.

That's an important distinction:

> **Same bucket does not mean same key.**

---

14. Collision Handling

Modern HashMap can use:

```
Linked List
```

or:

```
Red-Black Tree
```

inside a bucket.

Initially:

```
bucket 5
   ↓
A → B → C
```

If the collision chain becomes large enough, it may become:

```
bucket 5
   ↓
   Red-Black Tree
```

This is called **treeification**.

---

15. Why Treeify?

Suppose one bucket has many entries:

```
A → B → C → D → E → F → G → H
```

Searching through a long linked list can approach:

```
O(n)
```

A balanced tree can provide approximately:

```
O(log n)
```

within that bucket.

So treeification improves worst-case collision behavior.

---

16. Treeification Threshold

A key constant is:

```
TREEIFY_THRESHOLD = 8
```

When a bucket's node count reaches the relevant threshold, HashMap may attempt to treeify the bucket.

But there's an important nuance:

> **Eight entries in a bucket does not automatically mean "convert immediately to a tree."**

HashMap also considers the overall table capacity.

---

17. Minimum Treeify Capacity

Another important constant:

```
MIN_TREEIFY_CAPACITY = 64
```

If the table is too small, HashMap prefers resizing rather than treeifying.

Conceptually:

```
Bucket becomes crowded
        ↓
```

```
Capacity < 64?
   ↙            ↘
 YES             NO
 ↓               ↓
Resize        Treeify
```

This distinction is frequently tested.

---

18. Why Resize Instead of Treeify?

A small table can naturally create collisions simply because there aren't enough buckets.

Example:

```
Table size = 16
Many entries
```

Rather than immediately creating trees, increasing the number of buckets can spread the entries out.

So:

```
Small table + collisions
        ↓
Resize first
```

is generally more efficient.

---

19. Untreeification

A tree bin doesn't necessarily remain a tree forever.

If entries are removed and the number falls sufficiently, HashMap can convert the tree back into a linked structure.

Important constant:

```
UNTREEIFY_THRESHOLD = 6
```

So conceptually:

```
Many collisions
     ↓
Tree
     ↓
```

```
Entries removed
     ↓
Small enough
     ↓
Linked structure
```

The exact transition logic includes implementation details beyond simply comparing one number.

---

20. Load Factor

Default load factor:

```
0.75f
```

This is extremely important.

Suppose:

```
capacity = 16
load factor = 0.75
```

Threshold:

```
16 × 0.75 = 12
```

When the number of entries reaches the resize threshold, HashMap expands.

---

21. What Does Load Factor Mean?

Load factor controls the trade-off between:

```
Memory
vs
Collision frequency
```

Lower load factor:

```
More buckets
↓
Fewer collisions
↓
More memory
```

Higher load factor:

```
Fewer buckets
↓
Potentially more collisions
↓
Less memory
```

The default `0.75` is a practical compromise.

---

22. Capacity vs Size vs Threshold

These are often confused.

Capacity

Number of buckets:

```
16
```

Size

Number of mappings currently stored:

```
10
```

Threshold

Approximate size at which resizing occurs:

```
capacity × loadFactor
```

Example:

```
capacity = 16
load factor = 0.75

threshold = 12
```

So:

```
capacity ≠ size ≠ threshold
```

---

23. What Happens During Resize?

Suppose:

```
capacity = 16
threshold = 12
```

and HashMap needs to resize.

The table becomes:

```
16 → 32
```

Generally, HashMap doubles its capacity.

The threshold is recalculated accordingly.

---

24. Why Double the Capacity?

Doubling gives HashMap a convenient property.

If:

```
old capacity = 16
new capacity = 32
```

an existing entry generally either:

```
stays at the same index
```

or:

```
moves by +16
```

This is determined by a particular hash bit.

This makes resizing more efficient than recomputing arbitrary bucket positions from scratch.

---

25. The Resize Bit Trick

Suppose old capacity:

```
16
```

new capacity:

```
32
```

The new bit being introduced is:

```
16
```

For each entry, conceptually:

```
(hash & oldCapacity) == 0
```

means:

```
same index
```

Otherwise:

```
old index + old capacity
```

This is one reason powers-of-two capacities are so useful.

---

26. Does Rehashing Mean `hashCode()` Is Called Again?

This is a subtle interview question.

During resize, HashMap generally **does not need to call the key's `hashCode()` again** for every entry.

The stored hash is already available in each node.

It uses the old hash and the capacity bit to determine the new position.

So don't casually say:

> "Every resize calls `hashCode()` again for every key."

That's not how modern HashMap resizing works.

---

27. HashMap `get()` Internals

Suppose:

```
map.get("Java");
```

Conceptually:

```
get(key)
   ↓
Calculate hash
```

```
        ↓
    Calculate bucket index
        ↓
    Check first node
        ↓
    hash matches?
        ↓
    equals()?
        ↓
    Found?
```

If the bucket is a tree:

```
    Search Red-Black Tree
```

If it's a linked structure:

```
    Traverse nodes
```

---

28. Why Both `hash` and `equals()`?

Suppose two keys have different hash codes.

Then they cannot be the same logical key in a correctly implemented hash-based Map.

So HashMap can first check:

```
    hash
```

before calling:

```
    equals()
```

This avoids unnecessary equality comparisons.

Conceptually:

```
    hash differs
        ↓
    Not the key
```

If hash matches:

```
    hash same
        ↓
```

```
Check equals()
```

---

29. The `equals()` / `hashCode()` Contract

This is one of the most important HashMap concepts.

If:

```
a.equals(b)
```

is:

```
true
```

then:

```
a.hashCode() == b.hashCode()
```

must be true.

But the reverse isn't required.

Two different objects can have the same hash:

```
hashCode() same
equals() false
```

That's a collision.

---

30. Wrong `hashCode()` Implementation

Suppose:

```
class Employee {

    int id;

    @Override
    public boolean equals(Object o) {
        ...
    }
}
```

but `hashCode()` isn't overridden consistently.

Then two logically equal Employees may have different hashes.

HashMap can place them into different buckets.

Result:

```
equals() says same
HashMap lookup says different
```

This violates the required contract.

---

31. Correct Implementation

Prefer:

```
@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof Employee other)) return false;

    return id == other.id;
}

@Override
public int hashCode() {
    return Integer.hashCode(id);
}
```

Now:

```
Same id
 ↓
equals() true
 ↓
same hash
```

---

32. Mutable Keys — Major Trap

Consider:

```
Map<Employee, String> map =
    new HashMap<>();

Employee e = new Employee(101);
```

```
map.put(e, "John");
```

Now change:

```
e.setId(999);
```

If `id` participates in `hashCode()`:

```
Old hash
   ↓
Bucket A
```

becomes:

```
New hash
   ↓
Bucket B
```

But the entry is physically still associated with its original bucket structure.

Now:

```
map.get(e)
```

may fail.

Therefore:

> **Don't mutate fields that participate in `equals()` / `hashCode()` while an object is being used as a HashMap key.**

---

33. Why HashMap Isn't Thread-Safe

Consider two threads:

```
Thread 1 → put()
Thread 2 → put()
```

Both may modify internal state concurrently.

Without appropriate synchronization, operations can interfere with each other and visibility/order guarantees aren't provided.

Therefore:

```
HashMap
```

is not a concurrent Map.

For concurrent access:

```
ConcurrentHashMap
```

is usually the appropriate choice.

---

34. HashMap vs ConcurrentHashMap

Feature	HashMap	ConcurrentHashMap
Thread-safe	❌	✅
Null key	One	❌
Null values	✅	❌
Concurrent operations	❌	✅
General use	Yes	Concurrent applications

We'll deep-dive into `ConcurrentHashMap` later.

---

35. Fail-Fast Iterator

HashMap iterators are generally **fail-fast**.

Example:

```
for (Integer key : map.keySet()) {

    if (key == 10) {
        map.remove(key);
    }
}
```

This can produce:

```
ConcurrentModificationException
```

because the Map was structurally modified outside the iterator.

---

36. Correct Iterator Removal

Use:

```
Iterator<Integer> iterator =
    map.keySet().iterator();

while (iterator.hasNext()) {

    Integer key = iterator.next();

    if (key == 10) {
        iterator.remove();
    }
}
```

The iterator knows about its own removal operation.

---

37. Fail-Fast Is Not Thread Safety

Very important:

> Fail-fast behavior does **not** make HashMap thread-safe.

The purpose is to detect some structural modifications during iteration.

It is a **best-effort bug-detection mechanism**, not a synchronization mechanism.

---

38. HashMap Doesn't Guarantee Ordering

Don't depend on:

```
HashMap
```

iteration order.

Example:

```
Map<Integer, String> map =
    new HashMap<>();

map.put(3, "C");
map.put(1, "A");
map.put(2, "B");
```

You cannot write code assuming the iteration will be:

```
3
1
2
```

or:

```
1
2
3
```

Neither ordering should be relied upon.

If you need insertion order:

```
LinkedHashMap
```

If you need sorted keys:

```
TreeMap
```

---

39. HashMap and `null`

HashMap permits:

```
one null key
```

```
multiple null values
```

Example:

```
map.put(null, "A");
map.put(null, "B");
```

Result:

```
null → B
```

Only one null key exists because keys are unique.

---

40. Why Does HashMap Allow Null but ConcurrentHashMap Doesn't?

HashMap can represent:

```
key → null
```

without needing concurrent atomic semantics.

ConcurrentHashMap deliberately disallows null because concurrent operations need a clear distinction between:

```
key absent
```

and:

```
key mapped to null
```

This simplifies its concurrent API semantics.

---

41. HashMap Constructor

Common forms:

```
new HashMap<>();
```

```
new HashMap<>(32);
```

```
new HashMap<>(32, 0.75f);
```

Parameters:

```
initialCapacity
loadFactor
```

---

42. Initial Capacity Is Not Always Allocated Immediately

An important modern implementation detail:

Creating:

```
new HashMap<>(100);
```

doesn't necessarily mean an array of 100 buckets is immediately allocated.

HashMap table allocation is lazy and occurs when needed, such as during the first insertion.

Also, requested capacities are normalized according to HashMap's power-of-two sizing rules.

---

43. Default Capacity

The commonly documented/default initial capacity is:

```
16
```

with:

```
load factor = 0.75
```

However, remember:

> The internal table is lazily allocated, so the backing array isn't necessarily created at constructor invocation.

---

44. Initial Capacity and Performance

If you know approximately how many entries will be stored, providing an appropriate initial capacity can reduce repeated resizing.

For example:

```
Map<Integer, Employee> map =
    new HashMap<>(1000);
```

can be useful when the expected size is known.

But don't blindly over-allocate.

More buckets mean more memory.

---

45. HashMap's Core Formula

This deserves memorization.

Hash spreading

```
h ^ (h >>> 16)
```

Bucket index

```
(hash & (n - 1))
```

Resize threshold

Conceptually:

```
capacity × loadFactor
```

Default load factor

```
0.75
```

Treeification

```
TREEIFY_THRESHOLD = 8
```

Minimum treeify capacity

```
MIN_TREEIFY_CAPACITY = 64
```

Untreeification threshold

```
UNTREEIFY_THRESHOLD = 6
```

These are **implementation constants in current OpenJDK-style HashMap implementations**, not universal guarantees of every Java implementation/version.

---

46. Complete `put()` Mental Model

When you call:

```
map.put(key, value);
```

think:

```
                  put()
                    ↓
                hash(key)
                    ↓
            spread hash bits
                    ↓
           bucket = hash & (n-1)
                    ↓
                table[bucket]
                    ↓
                    ┴
               ┌────┴─────┐
               ↓          ↓
            empty      occupied
               ↓          ↓
           insert     compare keys
                          ↓
                          ┴
                    ┌─────┴──────┐
                    ↓            ↓
                 same key     different key
                    ↓            ↓
               replace value  collision chain
                                 ↓
                            list or tree
```

Then:

```
size > threshold
       ↓
    resize
```

---

47. Complete `get()` Mental Model

```
                get(key)
                   ↓
               calculate hash
                   ↓
             calculate bucket
                   ↓
               table[index]
                   ↓
              check candidate
                   ↓
             hash + equals()
                   ↓
          ┌────────┴────────┐
          ↓                 ↓
       found             not found
          ↓                 ↓
     return value         null
```

---

48. Why HashMap Is Fast

HashMap avoids searching every entry.

Instead:

```
Key
 ↓
Hash
 ↓
Bucket
 ↓
Small number of candidates
 ↓
Equality check
```

With a good hash distribution:

```
O(1) average
```

That's the fundamental reason HashMap is so widely used.

---

49. Poor Hash Distribution

Suppose every key returns:

```
@Override
public int hashCode() {
    return 1;
}
```

Then:

```
A → bucket X
B → bucket X
C → bucket X
D → bucket X
...
```

Everything goes into one bucket.

HashMap loses much of its expected performance advantage.

With modern treeification, sufficiently large collision bins can become trees, but a pathological hash function is still a poor design.

---

50. Same Hash ≠ Same Key

This is a classic interview question.

Suppose:

```
A.hashCode() = 100
B.hashCode() = 100
```

Does HashMap treat A and B as the same key?

No.

It checks:

```
hash
+
equals()
```

If:

```
A.equals(B) == false
```

both entries can coexist.

51. `HashMap` and Object Identity

HashMap normally uses logical equality:

```
equals()
```

not reference identity:

```
==
```

Therefore:

```
new String("Java")
```

and:

```
new String("Java")
```

can represent the same logical key if `equals()` returns true and their hashes agree.

For identity-based key semantics, Java provides:

```
IdentityHashMap
```

---

52. Why `containsKey()` Is Better Than `get() != null`

Suppose:

```
map.put("A", null);
```

Then:

```
map.get("A") == null
```

is true.

But the key exists.

Therefore:

```
map.containsKey("A")
```

is the correct test when you specifically want to know whether the key exists.

---

53. `putIfAbsent()` vs `put()`

```
map.put("A", 10);
```

always updates the mapping.

```
map.putIfAbsent("A", 20);
```

doesn't overwrite an existing mapping under the method's absent semantics.

So:

```
put()
→ insert or replace

putIfAbsent()
→ insert only when absent
```

---

54. HashMap and Java 7 vs Java 8+

This is a common senior-level interview topic.

Older Java implementations

Collision buckets were linked structures.

Heavy collisions could lead to poor worst-case lookup behavior.

Java 8+

HashMap introduced treeification of sufficiently large collision bins.

Conceptually:

```
Java 7-era
bucket → linked list

Java 8+
bucket → linked list
              ↓
       sufficiently crowded
              ↓
       Red-Black Tree
```

This significantly improves pathological collision behavior.

---

55. Why Treeification Was Introduced

Without treeification:

```
Bucket:
A → B → C → D → E → F → G → H
```

Searching could approach:

```
O(n)
```

With a balanced tree:

```
O(log n)
```

for the bin.

This protects HashMap from severe collision chains.

---

56. Does HashMap Always Use Red-Black Trees?

No.

Only sufficiently crowded bins can become tree bins.

Most normal buckets are either:

```
empty
```

or:

```
single node
```

or:

```
small linked structure
```

Tree bins are for unusually collision-heavy situations.

---

57. Memory Model

HashMap has memory overhead from:

```
table array
+
Node objects
```

```
+
references
+
hash values
```

Compared with a simple array, it's substantially more memory-intensive.

But that trade-off buys fast key-based lookup.

---

58. HashMap vs ArrayList Lookup

Suppose you need:

```
Find Employee with ID 5000
```

ArrayList:

```
iterate through elements
```

Potentially:

```
O(n)
```

HashMap:

```
employeeId
    ↓
hash
    ↓
bucket
    ↓
employee
```

Average:

```
O(1)
```

This is the core use case for HashMap.

---

59. Real-World Example

Suppose:

```
Map<Long, Employee> employees =
    new HashMap<>();
```

You receive:

```
employeeId = 10025
```

Then:

```
Employee employee =
    employees.get(10025L);
```

This is much more appropriate than repeatedly scanning a List when lookup by ID is the primary operation.

---

60. ⭐ Interview Scenario — Collision

Interviewer:

> Two different keys have the same hash code. What happens?

Strong answer:

> They can land in the same bucket. HashMap then distinguishes them using key equality. The bucket may use a linked structure, and if collisions become sufficiently large and the table is large enough, the bin can be treeified into a Red-Black Tree.

---

61. ⭐ Interview Scenario — Same Hash

Interviewer:

> If two keys have the same hash code, are they considered duplicate?

Answer:

> No. A hash collision only means they map to the same bucket. HashMap still uses equality to determine whether they represent the same key.

---

62. ⭐ Interview Scenario — Mutable Key

Interviewer:

> Why shouldn't you use a mutable object as a HashMap key?

Answer:

> If fields used by `equals()` or `hashCode()` change after insertion, the key's calculated bucket can change while the entry remains in its original location, making lookup or removal unreliable.

---

63. ⭐ Interview Scenario — Load Factor

Interviewer:

> Why is the default load factor 0.75?

Answer:

> It is a practical balance between memory usage and collision frequency. Lower values use more memory but generally reduce collisions; higher values save memory but can increase collisions.

---

64. ⭐ Interview Scenario — Power of Two

Interviewer:

> Why does HashMap use power-of-two capacities?

Answer:

> Because bucket selection can efficiently use `(n - 1) & hash`, and resizing by doubling gives an efficient redistribution rule where entries generally stay at the same index or move by the old capacity.

---

65. ⭐ Interview Scenario — Treeification

Interviewer:

> Why doesn't HashMap immediately convert a bucket into a tree when collisions begin?

Answer:

> Because small collision chains are cheap, and when the overall table is small, resizing can distribute entries more effectively. HashMap therefore uses a minimum capacity threshold before treeification.

---

66. ⭐ Interview Scenario — HashMap Thread Safety

Interviewer:

> Can multiple threads safely modify a HashMap?

Answer:

> Not without external synchronization or another concurrency mechanism. HashMap itself is not thread-safe. For concurrent access, `ConcurrentHashMap` is usually the appropriate implementation.

---

67. ⭐ Coding Question

What happens?

```
Map<String, Integer> map =
    new HashMap<>();

map.put("A", 10);
map.put("A", 20);

System.out.println(map.size());
System.out.println(map.get("A"));
```

Output:

```
1
20
```

Because the second `put()` replaces the value for the same logical key.

---

68. ⭐ Coding Question — Collision

Suppose:

```
class Key {

    int id;

    @Override
    public int hashCode() {
        return 1;
    }
```

```
        @Override
        public boolean equals(Object obj) {
            Key other = (Key) obj;
            return id == other.id;
        }
    }
```

Then:

```
map.put(new Key(1), "A");
map.put(new Key(2), "B");
```

Can both entries exist?

Yes.

Both have:

```
hashCode = 1
```

but:

```
Key(1).equals(Key(2)) == false
```

Therefore they are different keys.

---

69. ⭐ Coding Question — Mutable Key

```
Employee e =
    new Employee(101);

map.put(e, "John");

e.setId(999);

System.out.println(
    map.get(e)
);
```

Don't assume this will return `"John"`.

If `id` determines `hashCode()`, changing it after insertion can make the lookup fail.

---

70. ⭐ Coding Question — Null

```
Map<String, String> map =
    new HashMap<>();

map.put(null, "A");
map.put(null, "B");

System.out.println(map.size());
System.out.println(map.get(null));
```

Output:

```
1
B
```

One null key; second insertion replaces its value.

---

71. HashMap Interview Cheat Sheet

Memorize this:

```
HashMap
│
├── Hash table
│
├── Key → hash
│
├── Hash spreading
│      h ^ (h >>> 16)
│
├── Bucket
│      (n - 1) & hash
│
├── Collision
│      Linked structure
│          ↓
│      Tree bin when appropriate
│
├── Default load factor
│      0.75
│
```

```
├── Treeify threshold
│      8
│
├── Minimum treeify capacity
│      64
│
├── Untreeify threshold
│      6
│
├── Resize
│      roughly doubles capacity
│
├── Average lookup
│      O(1)
│
└── Not thread-safe
```

---

72. The Most Important HashMap Formula

If the interviewer asks:

> "How does HashMap find the bucket?"

Answer:

```
int hash = key.hashCode();
hash = hash ^ (hash >>> 16);

int index = (n - 1) & hash;
```

Conceptually, that's the important flow.

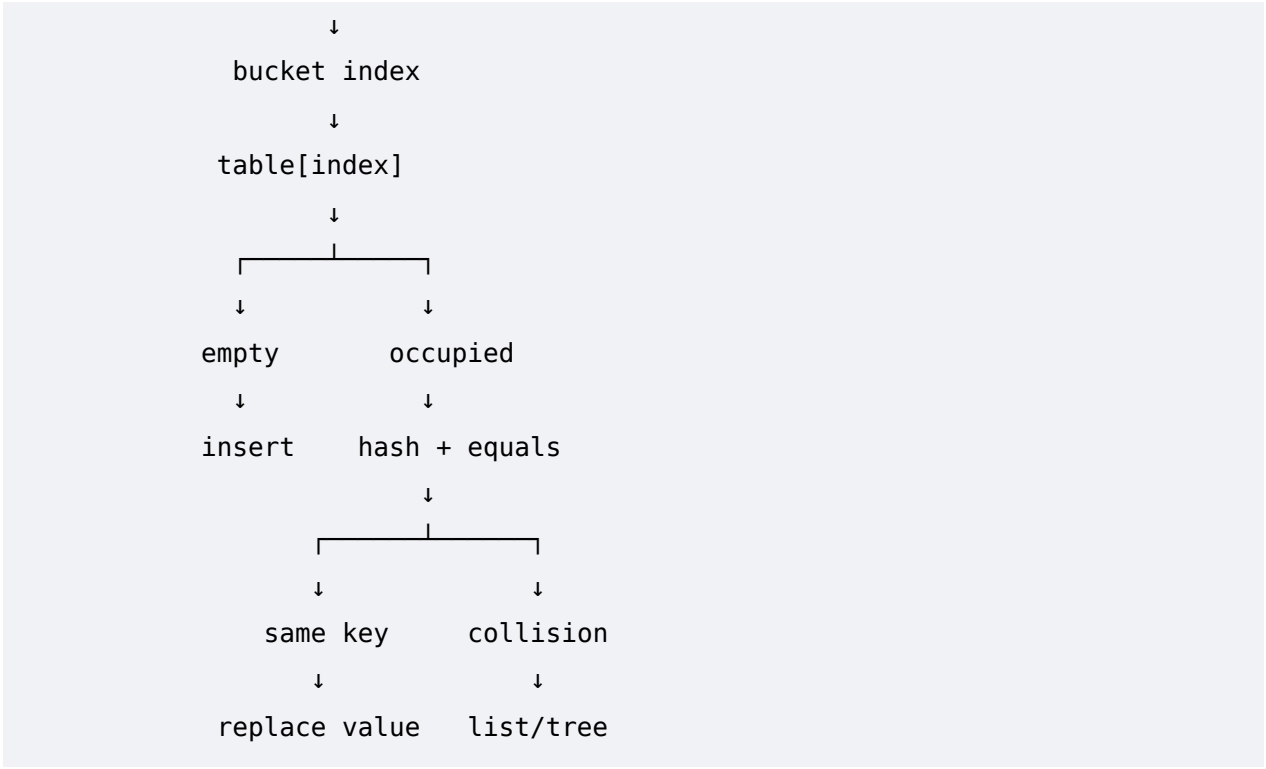
Don't overstate it as literally the complete implementation for every Java version.

---

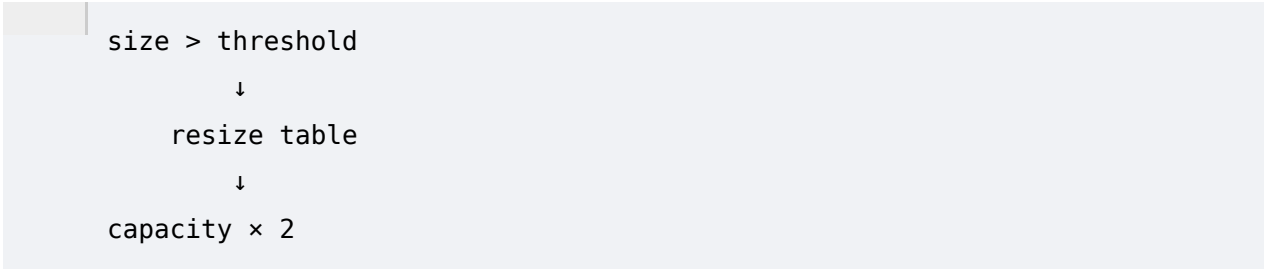
73. The Most Important HashMap Flow

Memorize:

```
              key
               ↓
         hashCode()
               ↓
         hash spreading
```



Resize:



---

74. HashMap vs LinkedHashMap vs TreeMap

You already know the basic distinction, but from an implementation perspective:

```
HashMap
→ Hash table
→ O(1) average
→ no ordering guarantee
```

```
LinkedHashMap
→ Hash table
→ linked ordering structure
→ O(1) average
→ predictable iteration order
```

```
TreeMap
→ Red-Black Tree
```

```
→ O(log n)
→ sorted keys
```

---

75. Final Decision Framework

```
Need key-value storage?
          ↓
         Map
          ↓
   ┌──────┼──────────┐
   ↓      ↓          ↓
 Fast   Insertion   Sorted
lookup   order      keys
   ↓      ↓          ↓
HashMap LinkedHashMap TreeMap
```

Concurrency:

```
Concurrent key-value access
          ↓
ConcurrentHashMap
```

Special cases:

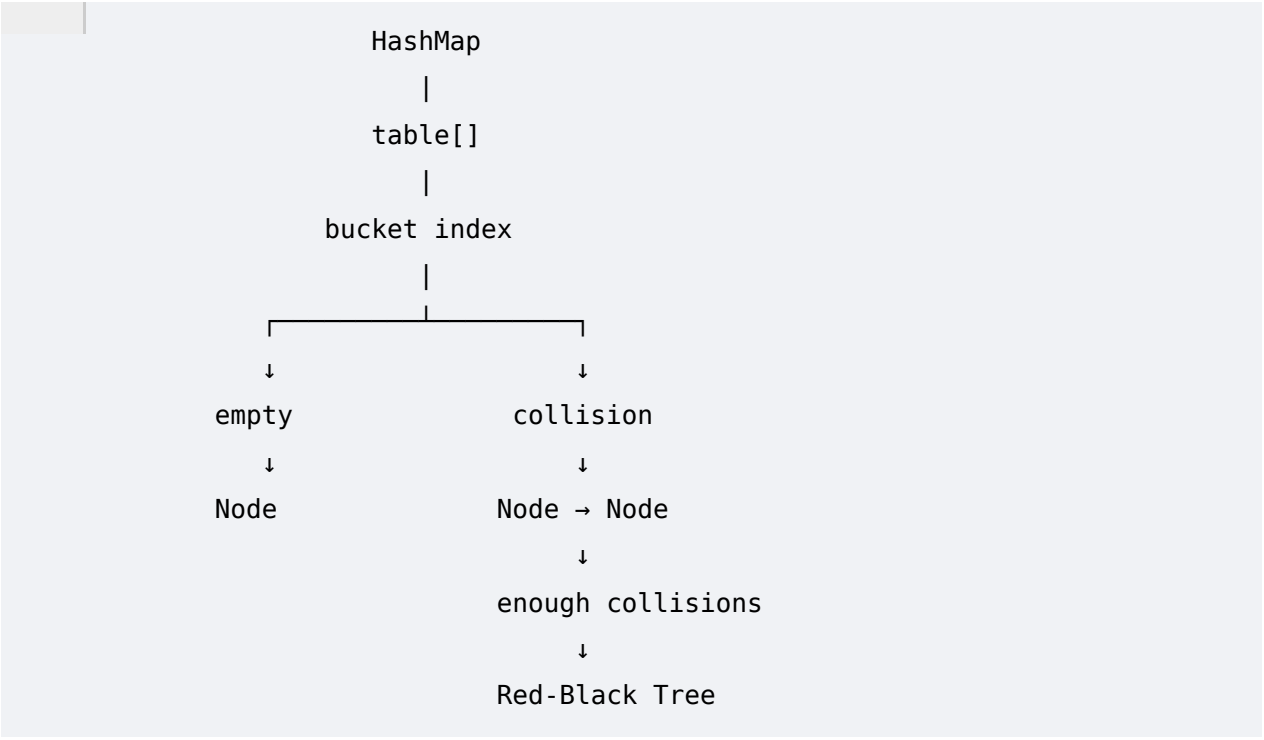
```
Enum keys
   ↓
EnumMap

Identity comparison
   ↓
IdentityHashMap

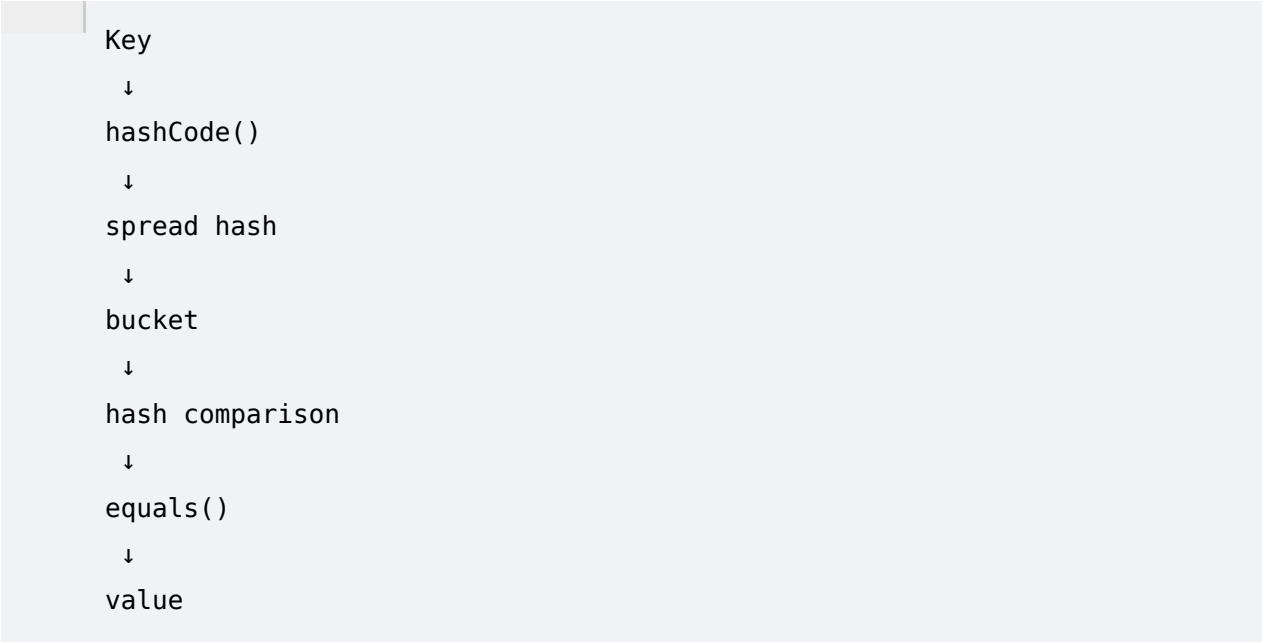
Weak key references
   ↓
WeakHashMap
```

🧠 Final Mental Model

Think of HashMap as:



And every lookup follows:



That sequence is the **heart of HashMap**.

🔥 Chapter 15 Summary

Concept	Key Point

Internal structure	Hash table
Storage	Array of buckets
Entry	`Node<K, V>`
Hash spreading	`h ^ (h >>> 16)`
Bucket index	`(n - 1) & hash`
Default capacity	16
Default load factor	0.75
Threshold	capacity × load factor
Collision	Multiple keys in same bucket
Collision structure	Linked nodes / tree bins
Treeify threshold	8
Minimum treeify	64

capacity	
Untreeify threshold	6
Resize	Capacity generally doubles
Average `get()`	O(1)
Average `put()`	O(1)
Tree-bin lookup	O(log n)
Null key	One allowed
Null values	Allowed
Ordering	No guarantee
Thread-safe	No
Iterator	Generally fail-fast

🎯 Senior-Level Interview Checklist

You should now be able to explain **without memorization gaps**:

- [ ] HashMap internal architecture

- [ ] What `Node<K,V>` contains
- [ ] `hashCode()` → hash spreading → bucket index
- [ ] Why `(n - 1) & hash` is used
- [ ] Why capacity is a power of two
- [ ] What a hash collision is
- [ ] Why same hash doesn't mean same key
- [ ] Role of `equals()`
- [ ] `equals()` / `hashCode()` contract
- [ ] HashMap `put()` flow
- [ ] HashMap `get()` flow
- [ ] Load factor
- [ ] Threshold
- [ ] Resizing
- [ ] Why capacity doubles
- [ ] What happens to entries during resize
- [ ] Treeification
- [ ] `TREEIFY_THRESHOLD`
- [ ] `MIN_TREEIFY_CAPACITY`
- [ ] `UNTREEIFY_THRESHOLD`
- [ ] Why HashMap isn't thread-safe
- [ ] Fail-fast iterators
- [ ] Mutable-key problem
- [ ] Java 7 vs Java 8+ collision handling
- [ ] HashMap vs LinkedHashMap
- [ ] HashMap vs TreeMap
- [ ] HashMap vs ConcurrentHashMap